

Sprawozdanie Algorytmów Dyskretnych

Lista 4

Jakub Kowal

1 Algorytm Edmondsa-Karpa

1.1 Implementacja

Moja implementacja algorytmu Edmondsa-Karpa działa w następujący sposób:

Edmonds-Karp 1:

```
Input  : źródło s, cel t
Output: Maksymalny możliwy przepływ
1 while True (faktycznie to dopóki jest ścieżka powiększająca z s do t) do
2   queue.push(s);
3   //Szukanie ścieżki poprzez BFS (zwraca minimalny łuk residualny)
   if Nie ma ścieżki powiększającej then
4     break;
5   else
6     //Przejdź znaną ścieżką powiększającą od końca (t) i
       powiększaj o znaleziony pathFlow
7   end
8 end
9 return maxFlow;
```

1.2 Złożoność

n — liczba węzłów.

m — liczba krawędzi.

Pętla wykona się najwyżej n razy bo każda ścieżka powiększająca wyczerpuje jedną z krawędzi, a ścieżki od s do t są ograniczone przez n , więc możemy wyczerpać najwyżej n ścieżek. $O(n)$

BFS jest $O(m)$ (ilość krawędzi)

Powiększanie przepływu jest $O(m)$, bo najdłuższa ścieżka powiększająca może być długości m .

Łącznie dostajemy $O(n \times m \times m)$, czyli $O(nm^2)$.

2 Algorytm Dinica

2.1 Implementacja

Dinic 2:

Input : źródło s , cel t

Output: Maksymalny możliwy przepływ

```
1 while bfsEstimate( $s, t$ ) (dopóki jest ścieżka między  $s$  a  $t$ ) do
2   while dfsPush( $s, t, max\_int$ ) > 0 (dopóki nie wyczerpały się ścieżki
   w grafie wartwowym stworzonym przez bfsEstimate( $s, t$ )) do
3     //Zliczamy łączną ilość ścieżek powiększających i sumę
     powiększonego przepływu.
4   end
5 end
6 return maxFlow;
```

2.2 Złożoność

Algorytm w każdej fazie wykona BFS o złożoności $O(m)$ oraz maksymalnie n DFS o złożoności $O(m)$. Wiemy, że graf nie będzie cykliczny, co oznacza, że po każdej fazie będziemy nasycać przynajmniej jedną krawędź. Uświadamia nam to, że faz może być najwyżej n (brak cykli w grafie). Więc ostatecznie otrzymujemy $O(n \times (m + n \times m))$, co daje nam: $O(n^2m)$.

3 Zapisywanie do modeli LP

3.1 Implementacja

Klasa FlowNetwork zwyczajnie zapisuje do pliku .jl najpierw wstęp potrzebny do modelu. Potem zapisuje ograniczenia przepływu wszystkich krawędzi (domyślnie jest 0) jako macierz kwadratową G . Jako zmienną tworzy macierz kwadratową f z ograniczeniami zapisanymi wcześniej w G . Następnie zapewnia ograniczeniami, że węzły inne niż s, t będą przyjmowały taki sam przepływ jaki przekazują dalej. Ostatecznie maksymalizujemy sumę przepływów wychodzących z pierwszego węzła, czyli naszego s .

4 Zadanie Hiperkostka

4.1 Opis zadania

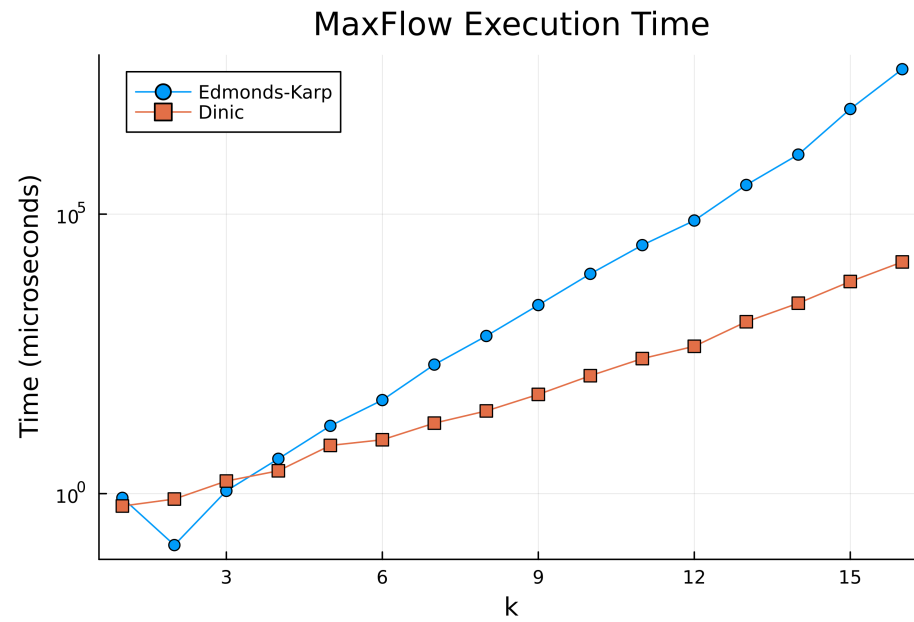
Dla parametru k zbudować skierowaną k -wymiarową hiperkostkę H_k o 2^k wierzchołkach opisanych k -bitowymi ciągami. Krawędzie łączą wierzchołki mające dokładnie jedną jedynekę w zapisie bitowym i skierowane są od wierzchołka o mniejszej wadze Hamminga do tego o większej wadze (mniejsza ilość jedynek

→ większa ilość jedynek). Pojemność każdej krawędzi (i,j) losowana jest równomiernie z $\{1, \dots, 2^l\}$, gdzie $l = \max\{H(e(i)), Z(e(i)), H(e(j)), Z(e(j))\}$ (H — waga Hamminga, Z — liczba zer ($k-H(e(i))$)).

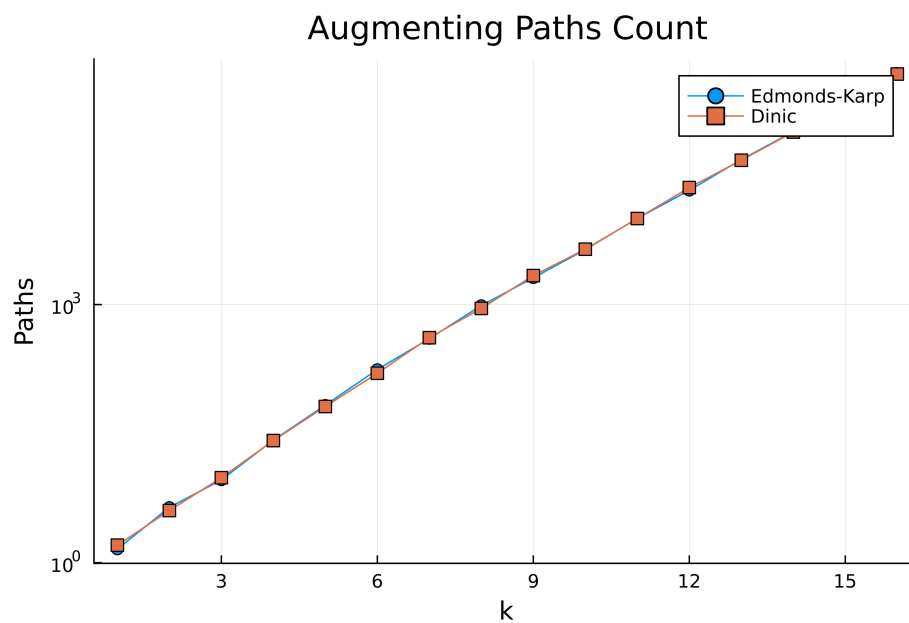
4.2 Wyniki

Wyniki są średnimi wartościami 25 powtórzeń.

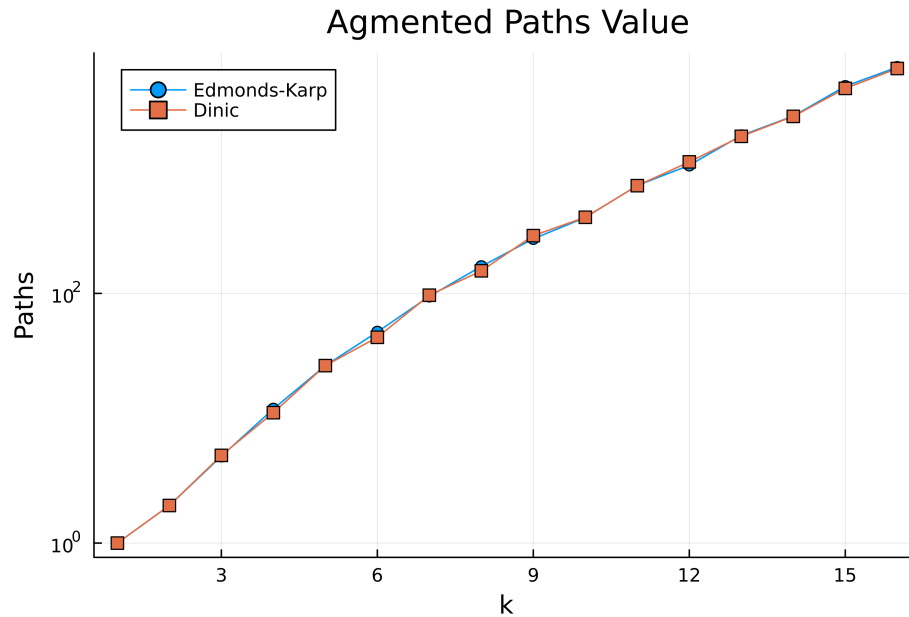
Rysunek 1: Wykres czasu względem k



Rysunek 2: Wykres ilości ścieżek powiększających względem k



Rysunek 3: Wykres maksymalnego przepływu względem k



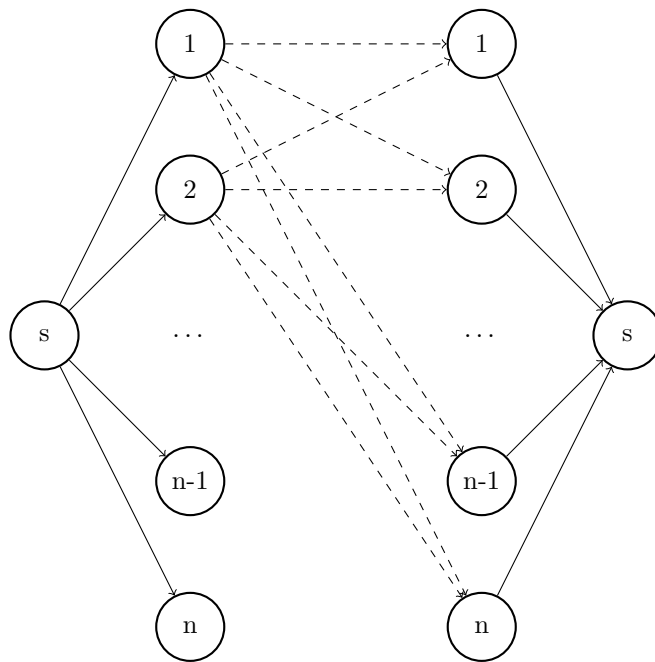
4.3 Wnioski

Jak widać na wykresie maksymalnego przepływu oraz wykresie ścieżek powiększających oba algorytmy zwracały niesamowicie zbliżone wyniki. Różnicę natomiast widać na wykresie czasu, jaki te algorytmy potrzebowały na rozwiązanie problemu. Oba algorytmy bardzo dobrze poradziły sobie z zadaniem, ale wyraźnie widać przewagę algorytmu Dinica.

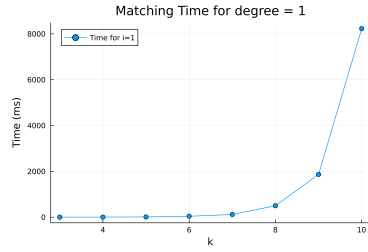
5 Zadanie Maksymalne Skojarzenia

5.1 Opis zadania

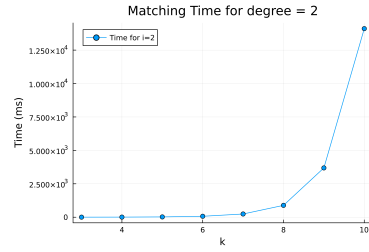
Prowizoryczna wizualizacja grafu:



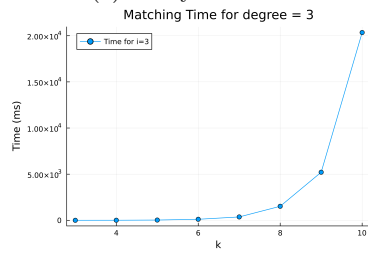
5.2 Wyniki



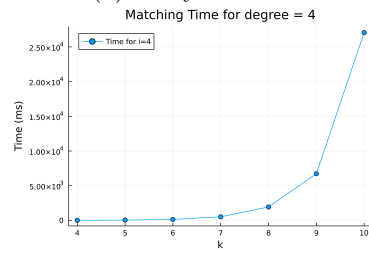
(a) Czasy dla $i = 1$



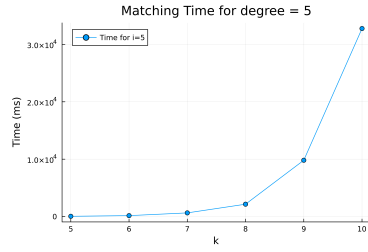
(b) Czasy dla $i = 2$



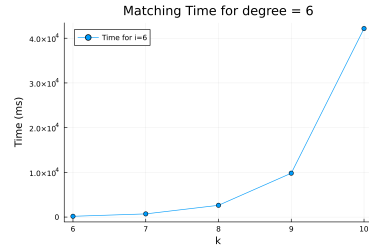
(c) Czasy dla $i = 3$



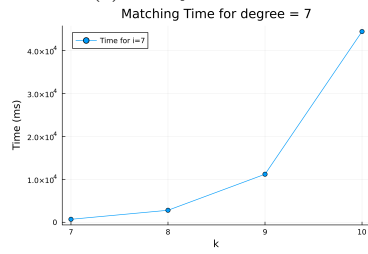
(d) Czasy dla $i = 4$



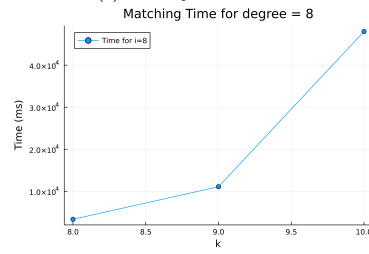
(e) Czasy dla $i = 5$



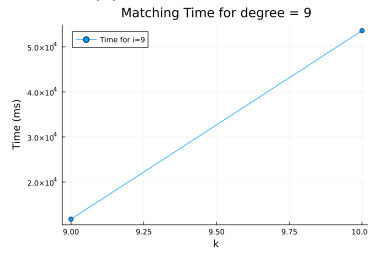
(f) Czasy dla $i = 6$



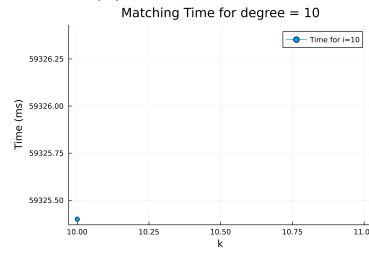
(g) Czasy dla $i = 7$



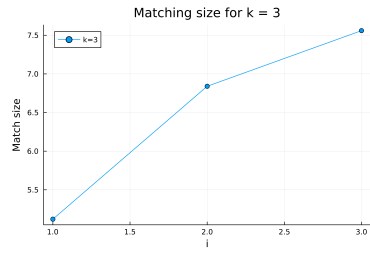
(h) Czasy dla $i = 8$



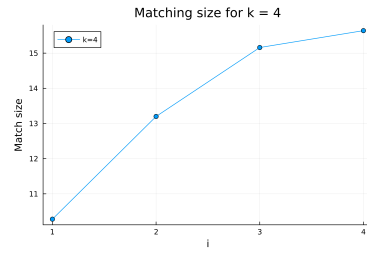
(i) Czasy dla $i = 9$



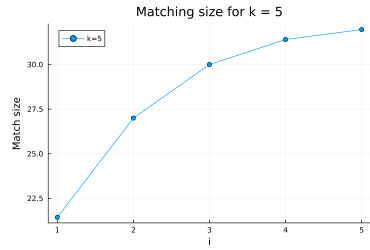
(j) Czasy dla $i = 10$



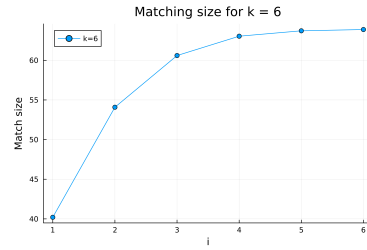
(a) Maksymalne skojarzenia dla $k = 3$



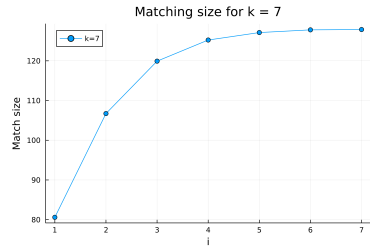
(b) Maksymalne skojarzenia dla $k = 4$



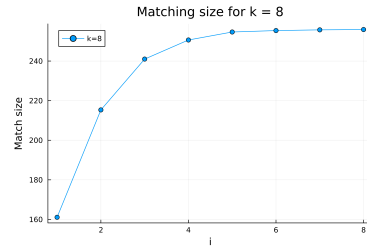
(c) Maksymalne skojarzenia dla $k = 5$



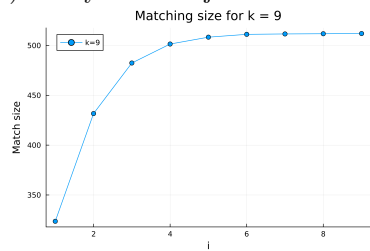
(d) Maksymalne skojarzenia dla $k = 6$



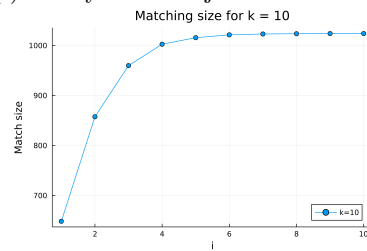
(e) Maksymalne skojarzenia dla $k = 7$



(f) Maksymalne skojarzenia dla $k = 8$



(g) Maksymalne skojarzenia dla $k = 9$



(h) Maksymalne skojarzenia dla $k = 10$

6 Porównanie c++ i Julii

6.1 Hiperkostka

k	JuMP Value	JuMP Time (s)	C++ Value	C++ Time (microseconds)
2	3.0	6.8233020305633545	3	2.0
3	14.0	6.6171910762786865	14	2.0
4	30.0	6.6985039710998535	30	8.0

Jak widać wyniki się zgadzają, ale c++ bez dwóch zdań pokonuje GLPK z Julii w szybkości. Dla tak niskich wartości k czas wykonywania w c++ musiał być mierzony w mikrosekundach, ponieważ algorytm wykonywał się krócej niż milisekunda. Jest to niewiarygodna prędkość względem solvera, który wykonywał się przez parę sekund.

6.2 Skojarzenia

k	JuMP Value	JuMP Time (s)	C++ Value	C++ Time (microseconds)
2	3.0	6.633029937744141	3	3.0
3	7.0	6.803858041763306	7	8.0
4	14.0	6.798024892807007	14	17.0

Jak można zauważyć na tabeli, wyniki znowu się pokrywają oraz c++ ponownie ma wielką przewagę w prędkości.